

Automated Logical Deciders

Research Notebook

Entry 001: Certificate-Indexed Halting, Independence Detection, and a Shared
Lemma Pool

Brian Tenneson

M.S., Applied Data Science

Currently Unaffiliated

Notebook status. This is a living research record, not a polished replacement for the full ALD paper. It records the motivating insight, working definitions, theorem sketches, literature findings, originality boundaries, frozen design decisions, experiment plans, and unresolved questions as of the opening entry. Future entries should preserve prior entries and append dated changes rather than silently rewriting the research history.

Started August 2, 2026, 9:55 AM Pacific

Copyright © 2026 Brian Tenneson. All Rights Reserved.

Contents

1 Notebook purpose and provenance	2
1.1 Connection to Version 47	2
2 Entry 001: the motivating obstruction	2
3 Working formal architecture	3
3.1 Typed evidence	3
3.2 The universally shared lemma pool	3
4 Initial theorem log	4
5 How independence can be detected now	5
5.1 Paired models	5
5.2 Dual finite saturation	5
5.3 Dual invariants	6
5.4 Relative consistency or formal metaproof	6
6 Why the shared lemma pool may change the problem	6
6.1 A necessary warning	6
7 Later machine-learning guidance	7
8 Originality research log	7
8.1 Search conducted for Entry 001	7
8.2 Closest prior art found	8
8.3 Provisional novelty boundary	8
9 Experiment notebook: proposed first studies	8
9.1 Experiment A: pool ablation	8
9.2 Experiment B: status ablation	9
9.3 Experiment C: synthetic independence benchmark	9
9.4 Experiment D: Predator integration	9
10 Frozen decisions as of Entry 001	9
11 Open questions for subsequent entries	10
12 Next-entry template	10
References consulted for Entry 001	12

1 Notebook purpose and provenance

This notebook begins after the observation that an Automated Logical Decider should not be modeled merely as two ATPs, one searching for a proof of c and one searching for a proof of $\neg c$. If both searches fail to halt, the computer has not detected independence. It has only failed to find either proof. The research question is therefore:

Opening research question

What finite, independently checkable evidence can license a multiagent system to report that a conjecture is proved, refuted, inconsistent with the base theory, independent of the base theory, or still unknown?

The second central observation is that the agents should not operate as isolated portfolios. Every agent should be able to add verified lemmas to a shared pool and use every compatible verified lemma at every round. This makes cooperation part of the mathematical object, not merely an implementation detail.

1.1 Connection to Version 47

Version 47 already distinguishes staged proof search, theorem birth, halting, finite consequence closure, preserved invariants, proof certificates, and bounded unknown outcomes. The present project reorganizes those ingredients around a richer terminal-status theory. The intended extension is not “an ATP that tries harder.” It is a certificate-indexed multiagent system whose different outcomes require different kinds of evidence.

2 Entry 001: the motivating obstruction

Fix an object theory $\mathcal{T}$, a conjecture c , and a declared metatheory $\mathcal{M}$ capable of checking claims about $\mathcal{T}$. Consider five roles:

1. a positive agent A_+ searching for $\mathcal{T} \vdash c$;
2. a negative agent A_- searching for $\mathcal{T} \vdash \neg c$;
3. a contradiction agent $A_\perp$ searching for $\mathcal{T} \vdash \perp$ or, more specifically, for derivations of both c and $\neg c$;
4. an independence agent A_I searching in $\mathcal{M}$ for certificates of $\mathcal{T} \not\vdash c$ and $\mathcal{T} \not\vdash \neg c$;
5. optional specialist agents for model finding, finite saturation, invariant discovery, relative consistency, proof translation, and certificate minimization.

The original obstruction is that A_+ and A_- may both run forever. Their silence has no unique logical interpretation. The replacement is to make nonderivability a positive search target in the metatheory.

Frozen conceptual distinction

Program termination, exhaustive search closure, and logical settlement are different. A timeout may stop a program; finite saturation may certify closure of a declared search space; only a status-appropriate certificate may settle a logical claim.

3 Working formal architecture

3.1 Typed evidence

Definition 3.1 (Evidence types). For an ALD instance $(\mathcal{T}, c, \mathcal{M})$, the basic evidence types are:

$$\begin{aligned} E_+ &: \mathcal{T} \vdash c, \\ E_- &: \mathcal{T} \vdash \neg c, \\ E_\perp &: \mathcal{T} \vdash \perp, \\ E_{nc} &: \mathcal{M} \vdash (\mathcal{T} \not\vdash c), \\ E_{n\neg c} &: \mathcal{M} \vdash (\mathcal{T} \not\vdash \neg c). \end{aligned}$$

An evidence ledger $\mathcal{E}_r$ records all verifier-accepted evidence available after round r .

Definition 3.2 (Status map). A conservative status map is defined from the ledger by:

$$\begin{aligned} \text{PROVED} & \quad \text{if } E_+ \in \mathcal{E}, E_-, E_\perp \notin \mathcal{E}, \\ \text{REFUTED} & \quad \text{if } E_- \in \mathcal{E}, E_+, E_\perp \notin \mathcal{E}, \\ \text{BOTH} & \quad \text{if } E_+, E_- \in \mathcal{E}, \\ \text{INCONSISTENT} & \quad \text{if } E_\perp \in \mathcal{E}, \\ \text{INDEPENDENT} & \quad \text{if } E_{nc}, E_{n\neg c} \in \mathcal{E} \text{ and no accepted inconsistency evidence defeats the declared interpretation,} \\ \text{UNKNOWN} & \quad \text{otherwise.} \end{aligned}$$

The ledger is more informative than a single label. In particular, if an inconsistent theory yields both c and $\neg c$, the system should preserve both proofs and the inconsistency evidence rather than overwrite one result with the other.

3.2 The universally shared lemma pool

Definition 3.3 (Provenance-aware lemma record). An object-level lemma record is a tuple

$$\lambda = (\ell, \Delta, \pi, o, r),$$

where ℓ is a formula, Δ is an explicit finite support set of assumptions, π is a verifier-accepted certificate of $\mathcal{T} \cup \Delta \vdash \ell$, o records the originating agent, and r is the first publication round. A metatheoretic record is defined analogously in $\mathcal{M}$.

Definition 3.4 (Round pool). Let $\mathcal{L}_r$ be the object-level pool and $\mathcal{K}_r$ the metatheoretic pool after round r . Every agent reads a snapshot of all compatible records in $\mathcal{L}_r \cup \mathcal{K}_r$ and may submit proposed additions during round $r + 1$. Only independently verified proposals are published.

Protocol 3.5 (Round-synchronous ALD). At each round r :

1. freeze a read-only snapshot $(\mathcal{L}_r, \mathcal{K}_r, \mathcal{E}_r)$;
2. allow every agent to perform one declared unit of search using every compatible record in the snapshot;
3. collect proposed lemmas and status certificates;
4. independently verify every proposal in a deterministic audit order;

5. publish accepted additions to form $(\mathcal{L}_{r+1}, \mathcal{K}_{r+1}, \mathcal{E}_{r+1})$;
6. apply the frozen halting policy to the new evidence ledger;
7. checkpoint the private agent states and global audit log.

Remark 3.6. The one-round delay is not logically necessary, but it makes causality, reproducibility, and race-condition auditing much cleaner. An asynchronous implementation can later be related to the synchronous model by recording a serializable publication order.

4 Initial theorem log

The following are working theorems for the ALD theory. Several are already formalized more fully in the accompanying paper; the notebook records why they matter and what remains to be sharpened.

Theorem 4.1 (Pool monotonicity). *For all r , $\mathcal{L}_r \subseteq \mathcal{L}_{r+1}$, $\mathcal{K}_r \subseteq \mathcal{K}_{r+1}$, and $\mathcal{E}_r \subseteq \mathcal{E}_{r+1}$.*

Proof. The protocol publishes only verified additions and never deletes an accepted record. Hence each structure is monotone by construction. $\square$

Theorem 4.2 (Object-pool soundness). *If every published object record $(\ell, \Delta, \pi, o, r)$ is accepted only when the object verifier confirms $\mathcal{T} \cup \Delta \vdash \ell$, then every use of that record in a branch whose assumptions include Δ is sound relative to $\mathcal{T}$ and that branch.*

Proof. The verifier establishes the recorded sequent. Weakening permits its use under any assumption context containing Δ . No other branch may treat ℓ as a theorem of $\mathcal{T}$ unless the support is discharged. $\square$

Corollary 4.3 (Safe universal visibility). *Every agent may see every pool record without compromising soundness, provided admissibility checks prevent an agent from using a record outside a context containing its recorded support.*

Theorem 4.4 (Support-discharge promotion). *Suppose $(\ell, \Delta, \pi, o, r) \in \mathcal{L}_r$ and for every $\delta \in \Delta$ the pool contains a verified proof of $\mathcal{T} \vdash \delta$. Then a verifier can construct a certificate of $\mathcal{T} \vdash \ell$, and ℓ may be promoted to a base-theory lemma with empty support.*

Proof. Substitute the verified derivations of the support assumptions into the certificate π and check the resulting composed derivation. $\square$

Theorem 4.5 (Status soundness). *Assume each status is emitted only after its designated sound verifier accepts the required evidence. Then every emitted conclusive status is correct relative to the declared object theory, metatheory, and status semantics.*

Proof. By cases on the status. For PROVED, REFUTED, and INCONSISTENT, the accepted object certificate supplies the corresponding derivation. For INDEPENDENT, the metatheoretic verifier accepts both nonderivability claims. The status map adds no stronger claim than the accepted evidence supports. $\square$

Theorem 4.6 (Unknown safety). *Resource exhaustion, heuristic stagnation, agent crash, or failure to locate a certificate may produce only UNKNOWN unless some conclusive evidence has already been accepted.*

Proof. These operational events are not proof objects for any conclusive logical claim. The frozen status policy therefore maps them to UNKNOWN. $\square$

Theorem 4.7 (Positive independence detection). *If the ALD accepts certificates π_c and $\pi_{\neg c}$ such that*

$$\text{Ver}_{\mathcal{M}}(\pi_c, \ulcorner \mathcal{T} \not\vdash c \urcorner) = 1 \quad \text{and} \quad \text{Ver}_{\mathcal{M}}(\pi_{\neg c}, \ulcorner \mathcal{T} \not\vdash \neg c \urcorner) = 1,$$

then the ALD may finitely emit INDEPENDENT without waiting for the positive and negative object-level searches to diverge forever.

Proof. The two accepted certificates are positive finite evidence for both nonderivabilities. The independence status follows by its definition. $\square$

Theorem 4.8 (Fair certificate discovery). *Fix a recursively enumerable class of finite certificates for each conclusive evidence type. Suppose every valid finite certificate is eventually proposed by at least one agent, every proposal is eventually checked, and the verifier terminates on every finite proposal. If a conclusive certificate exists in one of the covered classes, then the ALD eventually accepts it and reaches the corresponding conclusive ledger state.*

Proof. The valid certificate appears after finitely many proposals by fairness. It is then checked after finitely many additional rounds and accepted. Ledger monotonicity preserves it thereafter. $\square$

Theorem 4.9 (No universal total ALD). *For any sufficiently expressive recursively axiomatized theory with undecidable consequence problem, there is no computable sound ALD that returns a correct conclusive status for every sentence.*

Proof idea. A total sound ALD deciding PROVED versus non-PROVED for every sentence would decide theoremhood, contradicting undecidability. Adding more agents and certificate classes can enlarge finite coverage but cannot remove the computability barrier. $\square$

5 How independence can be detected now

The crucial move is to search for finite witnesses of nonderivability rather than interpreting nontermination. Several certificate families can support this.

5.1 Paired models

If the logic is sound and complete and the ALD produces:

$$\mathfrak{M}_+ \models \mathcal{T} \cup \{c\}, \quad \mathfrak{M}_- \models \mathcal{T} \cup \{\neg c\},$$

then neither c nor $\neg c$ follows from $\mathcal{T}$. This is the semantic pattern represented by the TPTP SZS status *NoConsequence*: one model supports the conjecture with the axioms and another supports its negation with the axioms.

5.2 Dual finite saturation

When a complete calculus on a finite effective domain reaches verified saturation for both $\mathcal{T} \cup \{c\}$ and $\mathcal{T} \cup \{\neg c\}$ without contradiction, those saturated states can serve as finite evidence of satisfiability, hence of nonderivability of the opposite sides under completeness assumptions.

5.3 Dual invariants

An invariant may exclude every derivation of c from $\mathcal{T}$, while another excludes every derivation of $\neg c$. The certificate must include proof that the initial state satisfies the invariant, every inference preserves it, and the excluded target violates it.

5.4 Relative consistency or formal metaproof

For deep independence results, the independence agent may search for a checked metaproof establishing

$$\text{Con}(\mathcal{T}) \Rightarrow \text{Con}(\mathcal{T} + c) \quad \text{and} \quad \text{Con}(\mathcal{T}) \Rightarrow \text{Con}(\mathcal{T} + \neg c),$$

or the appropriate stronger statement. The formal Lean proof of the independence of CH demonstrates that forcing-based independence reasoning can itself be kernel-checked, although automating the discovery of such arguments remains far beyond ordinary ATP search.

Approximately 100-word solution statement

The old method did not detect independence: two ATPs searched for c and $\neg c$, and if neither proof existed both could run forever. The ALD replaces silence with positive evidence. A dedicated independence agent searches in a declared metatheory for two finite, checkable certificates: one proving that c is not derivable from $\mathcal{T}$, and one proving that $\neg c$ is not derivable from $\mathcal{T}$. These certificates may be paired models, exhaustive finite saturations, dual invariants, relative-consistency arguments, or a checked formal metaproof such as forcing. Once both certificates verify, the ALD halts with **INDEPENDENT**. If they are not found, the result remains **UNKNOWN**.

6 Why the shared lemma pool may change the problem

The pool permits cross-status cooperation. A lemma discovered while trying to prove c may simplify the search for $\neg c$, expose a contradiction, help construct a model, or become a premise in a metatheoretic nonderivability proof. Conversely, a model-finding agent may identify a structural invariant that prunes both proof searches. This creates several research hypotheses.

Conjecture 6.1 (Cross-status reuse advantage). There exist natural benchmark families for which a universally shared verified lemma pool yields asymptotically fewer settlement expansions than every architecture using the same agents with role-isolated pools.

Conjecture 6.2 (Independence-agent acceleration). On some independent instances, lemmas generated by the positive and negative proof agents substantially reduce the cost of constructing the two nonderivability certificates, even though neither object-level proof search can succeed.

Conjecture 6.3 (Contradiction-agent value). A contradiction agent that consumes the shared pool can detect hidden inconsistency earlier than a portfolio that merely waits to see whether both c and $\neg c$ are independently proved.

6.1 A necessary warning

Unlabeled sharing is unsound. If the positive agent temporarily assumes c and proves ℓ , the negative agent may not treat ℓ as a theorem of $\mathcal{T}$. The pool must preserve support and logical level. This provenance requirement is therefore load-bearing, not administrative.

7 Later machine-learning guidance

Machine learning should be introduced only after the symbolic protocol and verifiers are frozen. The learned component may rank:

- which agent receives the next expansion budget;
- which open goal or branch is expanded;
- which assertion or inference rule is tried;
- which pool lemmas are retrieved for a given agent state;
- which proposed lemmas are likely to have cross-agent utility;
- which certificate family appears promising for the current instance.

Theorem 7.1 (Guidance-neutral soundness). *Replacing a hand-designed search policy by a learned policy does not change the soundness of accepted ALD statuses, provided the learned component cannot bypass the object verifier, metatheoretic verifier, support checker, or status map.*

Proof. The learned policy changes only proposal order and resource allocation. Every accepted lemma and status certificate still passes the same fixed verification conditions. $\square$

Remark 7.2 (Fairness protection). Pure learned ranking may starve a necessary low-ranked certificate path forever. A practical ALD should reserve a nonzero exploration schedule, such as round-robin expansions, stochastic exploration, or an exhaustive dovetailing component, so that finite-certificate completeness is not destroyed by the learned policy.

Conjecture 7.3 (Cross-agent utility learning). Training on first-use events across distinct agents yields a more transferable lemma-utility signal than training only on membership in final proofs, because it measures whether a lemma changes another search process rather than merely whether it appears in one completed certificate.

8 Originality research log

8.1 Search conducted for Entry 001

The opening literature review checked the following neighboring areas:

1. result ontologies and positive statuses for nonconsequence;
2. distributed and multiagent theorem proving;
3. blackboard architectures with globally visible proof states;
4. proof and model certificates;
5. formalized independence proofs;
6. reusable lemma pools and lemma mining;
7. machine-learned inference guidance.

8.2 Closest prior art found

Neighboring idea	What the literature already contains
Positive nonconsequence status	The TPTP SZS ontology defines <i>NoConsequence</i> : some models of the axioms satisfy the conjecture and some satisfy its negation. It explicitly identifies a pair of models or saturations as output evidence.
Multiagent blackboard proving	Leo-III was designed with reasoning agents cooperating through a shared blackboard at term, clause, and search levels. Earlier work describes globally shared data visible to all agents.
Distributed clause exchange	Clause-Diffusion and Aquarius use concurrent deductive processes that partition search and exchange clauses; modified variants address fairness and proof reconstruction.
Lemma pools and ML	Large-theory ATP research mines millions of intermediate lemmas, reuses them across later problems, and combines them with learned relevance filtering.
Learned inference ranking	ENIGMA learns clause-selection guidance inside saturation-based theorem provers while the symbolic prover retains inference correctness.
Kernel-checked independence	The independence of CH has been formalized in Lean using Boolean-valued models and forcing for both directions.

8.3 Provisional novelty boundary

The individual ingredients are not new. The strongest currently defensible originality claim is narrower:

Candidate-original synthesis

The reviewed sources did not reveal an established theory combining all of the following in one formal object: status-indexed object and metatheoretic agents for proof, negation-proof, contradiction, and paired nonderivability; a monotone evidence ledger separating stopping, exhaustion, and settlement; and a universally readable and writable lemma pool whose entries are independently verified, typed by logical level, and labeled by assumption support at every round.

This is not proof of worldwide novelty. A broader systematic review, citation chaining, expert consultation, and peer review are still required. In particular, multiagent blackboard systems come close on shared state, and the SZS ontology comes close on the semantic status taxonomy. The likely contribution is the *certificate-indexed SIC-style synthesis and theorem package*, not the invention of cooperation or model-pair independence evidence.

9 Experiment notebook: proposed first studies

9.1 Experiment A: pool ablation

Run the same frozen agents on the same theorem set under:

1. isolated private lemma pools;
2. a shared pool without ML;

3. a shared pool with provenance-aware learned retrieval.

Measure settlement rate, expansions, wall time, memory, accepted lemmas, cross-agent first uses, and certificate type.

9.2 Experiment B: status ablation

Compare:

1. proof-only ATP;
2. dual proof/negation ATP;
3. proof/negation/contradiction ALD;
4. full ALD including nonderivability certificate agents.

The main question is not merely how many proofs are found, but how many instances receive a logically justified finite status rather than UNKNOWN.

9.3 Experiment C: synthetic independence benchmark

Construct finite formal systems where independence is known by design and where paired saturation or invariant certificates are short. This permits testing of ALD mechanics before attempting forcing-level mathematics.

9.4 Experiment D: Predator integration

Treat Predator variants as proposal agents under one common verifier. Begin with the locked 50-theorem benchmark for theorem-proving performance, then build a separate ALD benchmark containing theorem, negation-theorem, inconsistent, finite-independent, and bounded-unknown instances. The theorem benchmark and ALD-status benchmark should not be conflated.

10 Frozen decisions as of Entry 001

D1: certificate requirement

No conclusive status is emitted solely because an agent stopped, timed out, exhausted its assigned local budget, or produced no new preferred actions.

D2: universal pool access

Every agent may contribute to and use the shared verified lemma pool at every round, subject to logical-level and assumption-support compatibility.

D3: verifier separation

Search agents, including learned agents, do not determine correctness. Independent object and metatheoretic verifiers determine acceptance.

D4: independence means paired nonderivability

The ALD reports independence only after accepting both $\mathcal{T} \not\vdash c$ and $\mathcal{T} \not\vdash \neg c$ relative to a declared metatheory and stated assumptions.

D5: unknown remains a legitimate result

Undecidability and certificate scarcity imply that some runs must remain UNKNOWN. The goal is to convert some previously silent cases into finite certified settlements, not to promise a universal decider.

11 Open questions for subsequent entries

Open Question 11.1 (Best status algebra). Should INCONSISTENT dominate all other statuses, or should the ALD always output the full evidence vector $(E_+, E_-, E_\perp, E_{nc}, E_{n-c})$ together with a derived human-readable label?

Open Question 11.2 (Minimal support). Can support minimization be performed efficiently enough that the pool records near-minimal assumptions, preventing unnecessary contamination of cross-agent reuse?

Open Question 11.3 (Metatheory hierarchy). Should the independence agent itself be an ALD operating in a stronger metatheory, producing a hierarchy $\mathcal{T}_0 \subseteq \mathcal{T}_1 \subseteq \dots$ of certificate-checking levels?

Open Question 11.4 (Cross-agent fairness). What is the weakest scheduler condition ensuring that no finite certificate path is permanently starved while still permitting aggressive learned allocation?

Open Question 11.5 (Originality). Which earlier systems combine model finding, proof search, and a shared blackboard with typed status evidence? A systematic citation search should focus on “cooperative decision procedures,” “proof and model portfolios,” “blackboard theorem proving,” “non-theorem certificates,” and “independence automation.”

Open Question 11.6 (Benchmark design). How should the first ALD benchmark balance object-theory proofs, refutations, contradictions, finite independence, semantic independence, and genuinely unresolved cases without leaking target-specific certificates into training?

12 Next-entry template

Future notebook entries should append:

1. date, time, and trigger;
2. new definitions or changes to old definitions;
3. theorem statements with proof status: proved, proof sketch, computationally tested, conjectured, or refuted;
4. literature searches and exact novelty boundaries;
5. experiments, frozen code/data hashes, and outcomes;
6. decisions that are now locked;

7. mistakes, failed approaches, and unresolved contradictions;
8. next actions.

References consulted for Entry 001

References

- [1] G. Sutcliffe, “The SZS Ontologies and Presentation Standards,” TPTP documentation, especially the *NoConsequence*, *Unknown*, *Timeout*, proof, model, and saturation statuses, accessed August 2, 2026.
- [2] M. P. Bonacina and J. Hsiang, “The Clause-Diffusion Methodology for Distributed Deduction,” *Fundamenta Informaticae*, 24(1–2):177–207, 1995. DOI: 10.3233/FI-1995-24128.
- [3] M. P. Bonacina, “On the Reconstruction of Proofs in Distributed Theorem Proving: A Modified Clause-Diffusion Method,” *Journal of Symbolic Computation*, 21(4–6):507–522, 1996. DOI: 10.1006/jsc.1996.0028.
- [4] M. Wisniewski and C. Benz Müller, “Is It Reasonable to Employ Agents in Automated Theorem Proving?” in *Proceedings of ICAART 2016*, vol. 1, pp. 281–286. DOI: 10.5220/0005824702810286.
- [5] C. Kaliszyk and J. Urban, “Learning-Assisted Theorem Proving with Millions of Lemmas,” *Journal of Symbolic Computation*, 69:109–128, 2015. DOI: 10.1016/j.jsc.2014.09.032.
- [6] C. Kaliszyk, J. Urban, and J. Vyskočil, “Lemmatization for Stronger Reasoning in Large Theories,” in *FroCoS 2015*, LNAI 9322, pp. 341–356. DOI: 10.1007/978-3-319-24246-0_21.
- [7] J. Jakubův and J. Urban, “ENIGMA: Efficient Learning-Based Inference Guiding Machine,” *arXiv:1701.06532*, 2017.
- [8] J. M. Han and F. van Doorn, “A Formal Proof of the Independence of the Continuum Hypothesis,” *arXiv:2102.02901*, 2021.